

* See NLP Section

⇒ RNN

* Can we solve "sequential Data" with ANN?

→ ANN Mostly/Best works on Tabular Data

e.g. Dataset: { sentiment Analysis }

Text	O/P
The <u>Food</u> is <u>good</u>	1
The <u>Food</u> is <u>Bad</u>	0
The <u>Food</u> is <u>not</u> good	0

Vocabulary (unique Words) = 4

→ Using Bag of Words: [See In NLP Section]

	<u>Food</u>	<u>good</u>	<u>bad</u>	<u>not</u>
S ₁ :	1	1	0	0
S ₂ :	1	0	1	0
S ₃ :	1	1	0	1

* Text Data $\Rightarrow$ Sequence information is important.

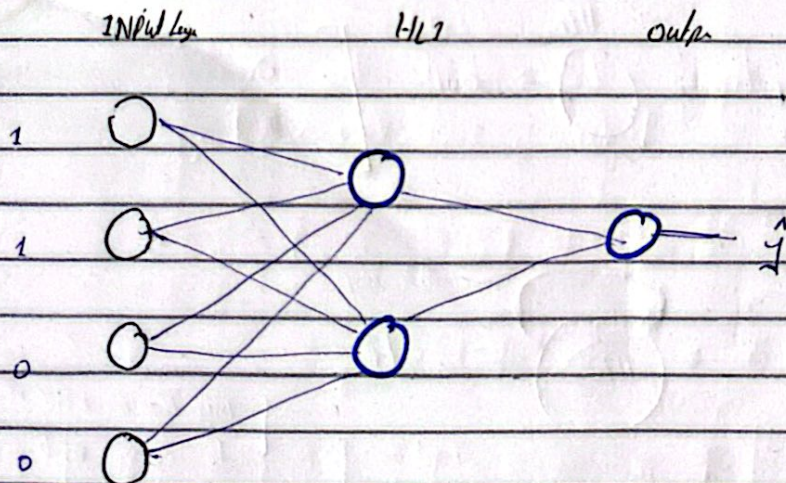
$\Rightarrow$ But in Bag of Words The words are not in sequence

$\Rightarrow$ ~~Sequence is lost and meaning~~

$\Rightarrow$ Sequence is lost, means meaning of sentence is lost.

X — X

	Food	good	but	not
S ₁ :	1	1	0	0



* Text Data: In Sequence of Info, we should give one word at a time.

at $t_1 \Rightarrow$ The, $t_2 \Rightarrow$ Food, $t_3 \Rightarrow$ is, $t_4 \Rightarrow$ Good

* But here we are give whole sentence at once which is not good.

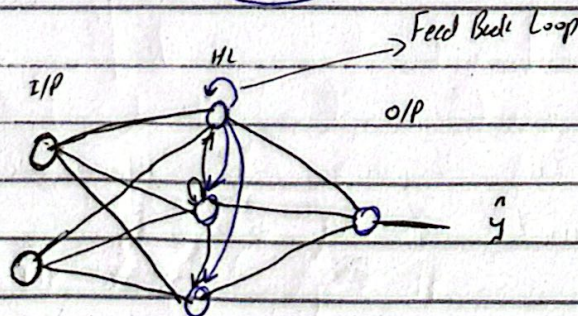
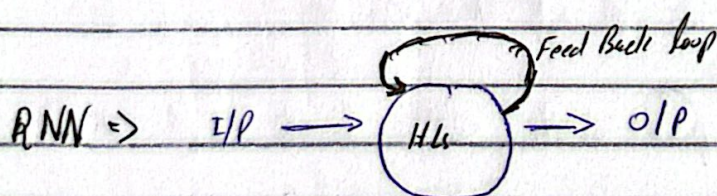
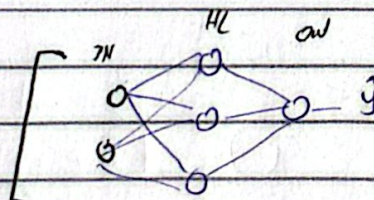
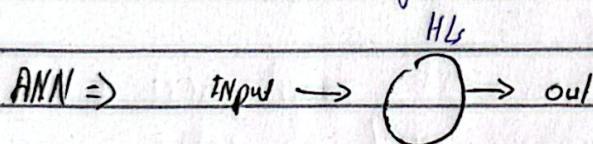
So we can't use ANN for Sequential Data.

- 1) Sequence is lost
- 2) Give whole sentence at once.

x — x

Recurrent Neural Network (RNN):

* Data is sequential



[This Node sends its output to every node in the Hidden Layer]

* * Each Node sends its output to every Node in the hidden Layer

⇒ The Food is Good

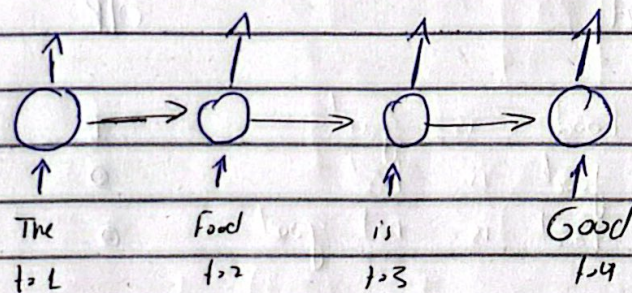
* In Case of RNN, we pass input at each time stamp.

$t=1$ ⇒ The

$t=2$ ⇒ Food

$t=3$ ⇒ is

$t=4$ ⇒ Good



at $t=3$ The Neural Network has Content of previous words as its connected.

⇒ Forward Propagation

Dataset:

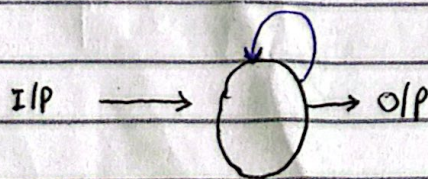
	Text	O/P
S1 ⇒	The Food is good	1
	The food is bad	0
	The Food is not good	0

* We need to convert these words into vectors [Bow, word2Vec, OHE]

⇒ Unique words (Vocabulary Size) ⇒ 5

⇒ Using OHE. ⇒ Vectors of Sentence 1

S1 ⇒ $\begin{bmatrix} [1, 0, 0, 0, 0], [0, 1, 0, 0, 0], [0, 0, 1, 0, 0] \\ [0, 0, 0, 1, 0], [0, 0, 0, 0, 1] \end{bmatrix}$



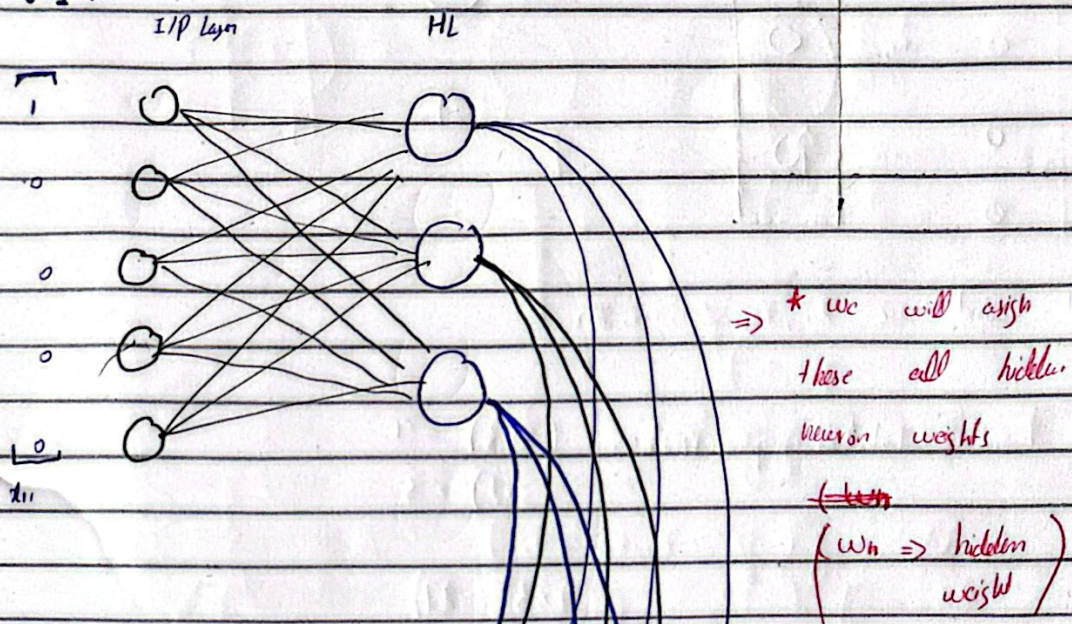
★ In S_1 I have 5 words, we will pass each word at a time

$$S_1 = \left[\begin{matrix} [10000] & [01000] & [00100] & [000100] & [00001] \end{matrix} \right]$$

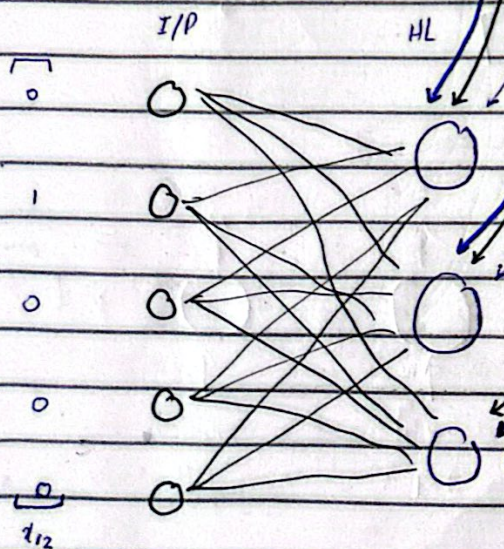
x_{11} x_{12} x_{13} x_{14} x_{15}

★ at each time timestamp the nodes are connected.

★ $t=1$:

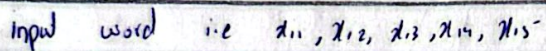


★ at $t=2$



★ at $t=3$ all the time stamps are connected like t_1 & t_2 like

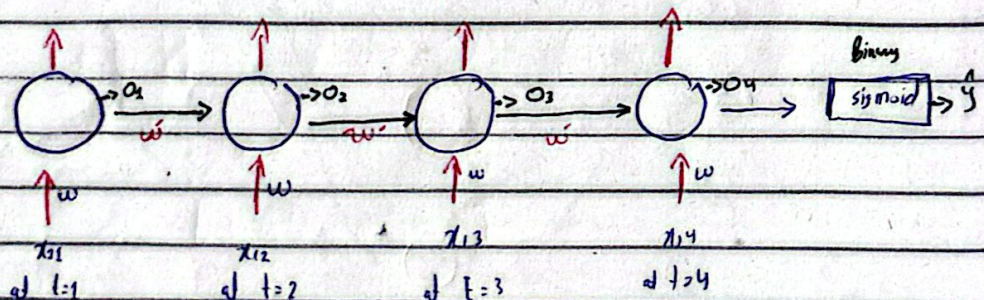
For every neuron (2×3) in entire n HL becomes $\frac{n}{3} \times 3$



$$9 \times 3 = 27$$

50

F.P



$$O_1 \Rightarrow f(x_1, \omega_1 + b_1), \quad O_2 \Rightarrow f(x_2, \omega_2 + b_2)$$

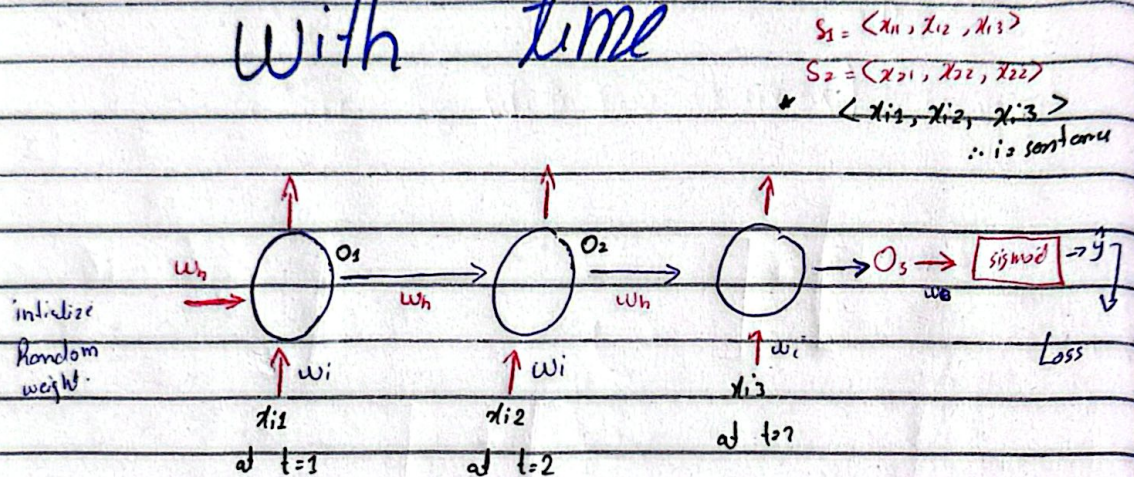
$$o_1 = f(x_{11} * \dot{w} + b)$$

$$o_2 = f(x_{12} * w + o_1 * \dot{w} + b)$$

$$o_3 = f(x_{13} * w + o_2 * \dot{w} + b)$$

$$o_4 = f(x_{14} * w + o_3 * \dot{w} + b)$$

⇒ RNN Back Propagation with time



$\star \star$ Forward Propagation.

$$o_1 = f(x_{i1} * w_{i1} + b_1)$$

$$o_2 = f(x_{i2} * w_{i2} + o_1 * w_{h1} + b_2)$$

$$o_3 = f(x_{i3} * w_{i3} + o_2 * w_{h2} + b_3)$$

$$\star \star \hat{y} = \underset{\text{sigmoid}}{\sigma}(o_3 * w_o)$$

$\text{Loss} = (y - \hat{y}) \Rightarrow$ Reduce this loss, updates all the weights

update $\Rightarrow [w_i, w_h, w_o]$

$\star$ We need to keep updating these weights in Back propagation.

$$\text{update} = [w_i, w_h, w_o]$$

★ Once we calculate Loss, The 1st weight we need to update is "w_o"

⇒ Weight update Formula:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial \text{Loss}}{\partial w_{\text{old}}}$$

★ ⇒ update w_o

$$\Rightarrow w_{o_{\text{new}}} = w_{o_{\text{old}}} - \eta \frac{\partial \text{Loss}}{\partial w_{o_{\text{old}}}}$$

$$\frac{\partial \text{Loss}}{\partial w_{o_{\text{old}}}} \Rightarrow \frac{\partial \text{Loss}}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_{o_{\text{old}}}}$$

★ Update w_h:

* hidden layer weight → imp b/c used in all time stamps.

$$\Rightarrow w_{h_{\text{new}}} = w_{h_{\text{old}}} - \eta \frac{\partial \text{Loss}}{\partial w_{h_{\text{old}}}}$$

at t=3:

$$\frac{\partial \text{Loss}}{\partial w_{h_{\text{old}}}} \Rightarrow \left[\frac{\partial \text{Loss}}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial o_3} * \frac{\partial o_3}{\partial w_{h_{\text{old}}}} \right] + \left[\frac{\partial \text{Loss}}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial o_2} * \frac{\partial o_2}{\partial o_2} \right] + \left[\frac{\partial \text{Loss}}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial o_3} * \frac{\partial o_3}{\partial o_2} * \frac{\partial o_2}{\partial o_1} * \frac{\partial o_1}{\partial w_{h_{\text{old}}}} \right]$$

at t=2

at t=1

Updating W_i :

* It is also based on time stamps:

$$\Rightarrow W_{i, \text{new}} = W_{i, \text{old}} - \eta \frac{\partial \text{Loss}}{\partial W_{i, \text{old}}}$$

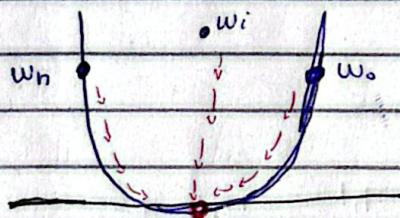
at $t=2$

at $t=3$

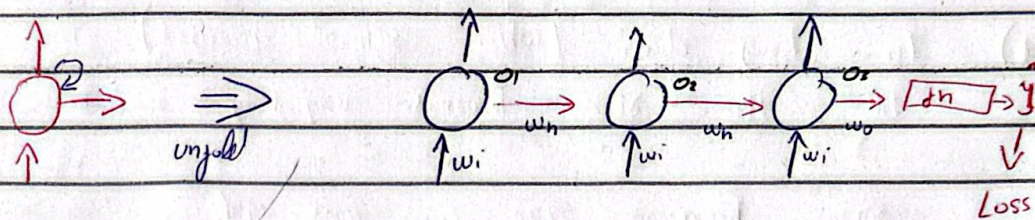
$$\frac{\partial \text{Loss}}{\partial W_{i, \text{old}}} \Rightarrow \left[\frac{\partial \text{Loss}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial O_3} \times \frac{\partial O_3}{\partial W_i} \right] + \left[\frac{\partial \text{Loss}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial O_3} \times \frac{\partial O_3}{\partial O_2} \right]$$

$$* \left[\frac{\partial O_2}{\partial W_i} \right] + \left[\frac{\partial \text{Loss}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial O_3} \times \frac{\partial O_3}{\partial O_2} \times \frac{\partial O_2}{\partial O_1} \times \frac{\partial O_1}{\partial W_i} \right]$$

at $t=1$



⇒ Problems with RNN



Text Generation:

⇒ I like to play — .

* I need to predict the last word, but this word may depend on [I, or like or to or play]

* Long Sentence:

⇒ ^{t_1} My ^{t_2} name ^{t_3} is ^{t_4} Alyan ^{t_5} and ^{t_6} I
am learning ML and DL, I
have also made project &
 ^{t_n} they — .

* The prediction may depend on t_1 or t_2 or t_n in the middle, but Long term dependency can't be captured using RNN.

(1) Long Term Dependency Can't be captured using RNN.

(when sentence gets longer & longer the performance drops).

2) In Long chain (when sigmoid is used) in BP the $\frac{\partial \text{loss}}{\partial w}$ becomes very, very smaller

approach becomes zero & we get vanishing gradient problem.

e.g 50 word

for t_1 = chain is from 50 till 1
 t_2 = from 49 till 2
 t_3 = from 48 till 3

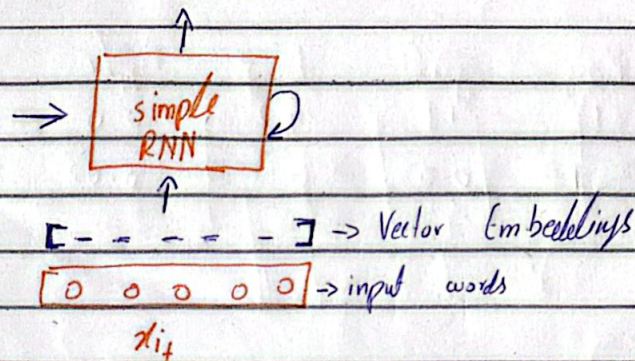
★ means The earlier words have very long chain & due to vanishing gradient will become zero & won't participate much in prediction of 150 word.

⇒ Word Embedding:

* Feature Representation

Input:

Test	O/P
$\langle x_{11}, x_{12}, x_{13}, x_{14} \rangle$	0
$\langle x_{21}, x_{22}, x_{23}, x_{24} \rangle$	1
- - - -	-



* We need to convert the input words into word Embeddings.

$\lambda \quad \text{---} \quad x$

Let's consider we are using:

ONE HOT:

Vocabulary size = 10,000

* each word will be represented by 10,000 dimension $\Rightarrow$ at only one place will be "one" (where the word is present), remaining will be zero

$$\text{man} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ \vdots \end{bmatrix} \quad \text{boy} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ \vdots \end{bmatrix} \quad \Rightarrow \quad \text{* Sparse Matrix leads to Overfitting.}$$

at this index man is present.

⇒ See NLP Section:

⇒ Word Embedding: [word 2 Vec] ⇐ Embedding Layer

* Vocab size = 10,000

Feature Representation:

* 300 Dimension → Represent each word with these 300 Dimension

I have these words.

Features	Boy	girls	king	Apple
Gender	-1	+1	-9.2	0.0
Royal	0.01	0.03	0.25	0.02
Age				
Food				

[see word 2 Vec Section]
[in NLP Notes]